

IASI Quarto User Guide

Framework for Engineering Documentation with Quarto

IASI Project

Table of contents

1	Introducción	3
1.1	iasi.quarto y Quarto	3
1.2	¿Qué encontrarás en este libro?	3
1.3	Requisitos	4
2	Quarto como base	5
2.1	Qué hace Quarto	6
2.2	Qué añade iasi.quarto	7
2.3	Separación de responsabilidades	8
2.4	Un proyecto sigue siendo un proyecto Quarto	9
2.5	Cuándo resulta útil	10
3	Qué es iasi.quarto	11
3.1	Una convención sobre Quarto	11
3.2	El problema que resuelve	11
3.3	Qué aporta	12
3.4	Qué no pretende ser	12
3.5	Una herramienta para proyectos que crecen	13
4	Fundamentos	14
4.1	La publicación IASI Quarto	14
4.1.1	La firma de una publicación	14
4.1.2	Una publicación no es necesariamente un libro	15
4.1.3	Publicación individual	16
4.1.4	Multiproyecto	16
4.1.5	La publicación actual tiene prioridad	17
4.1.6	Publicación y contenido	18
4.1.7	Archivos fuente y archivos derivados	19
4.1.8	Reconocer no significa construir	20
4.1.9	La publicación como unidad de construcción	20

1 Introducción

Bienvenido a la guía de usuario de **iasi.quarto**.

Este libro está dirigido a quienes desean utilizar **iasi.quarto** para construir proyectos documentales basados en Quarto.

No es necesario conocer la arquitectura interna del framework ni los detalles de su implementación.

El objetivo de esta guía es ayudarte a comprender la filosofía de **iasi.quarto**, instalarlo, crear tu primer proyecto y aprovechar sus capacidades en situaciones reales.

1.1 iasi.quarto y Quarto

iasi.quarto vive sobre Quarto.

No pretende sustituirlo, modificar su modelo documental ni crear un sistema de publicación alternativo.

Quarto continúa siendo el motor responsable de procesar los documentos y generar los formatos finales. **iasi.quarto** proporciona una capa de organización, automatización y convenciones que facilita el desarrollo y mantenimiento de proyectos documentales de cualquier tamaño.

1.2 ¿Qué encontrarás en este libro?

A lo largo de esta guía aprenderás a:

- comprender el propósito de **iasi.quarto**;
- instalar el framework;
- crear un nuevo proyecto;
- organizar la documentación;
- generar libros en distintos formatos;
- configurar el comportamiento del framework;
- utilizar la API pública;
- resolver los problemas más habituales.

Cada capítulo desarrolla un aspecto concreto y puede consultarse de forma independiente cuando ya se conoce el funcionamiento básico del framework.

1.3 Requisitos

Se recomienda tener conocimientos básicos de:

- Quarto;
- Git;
- R (para utilizar el paquete `iasi.quarto`).

No obstante, los conceptos específicos del framework se explican desde el principio.

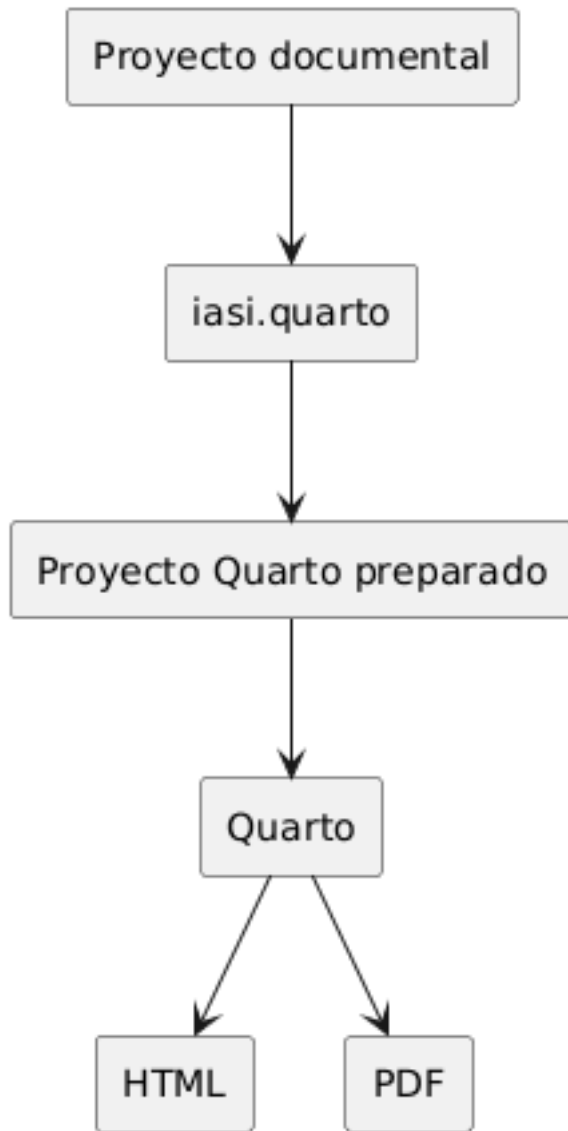
Comencemos entendiendo por qué nació el framework y qué problemas pretende resolver.

2 Quarto como base

`iasi.quarto` está construido sobre Quarto.

No introduce un nuevo formato documental ni sustituye el proceso de publicación de Quarto. Los archivos continúan siendo documentos `.qmd`, la configuración sigue expresándose mediante YAML y los formatos finales son generados por Quarto.

Esta relación permite utilizar `iasi.quarto` sin abandonar el ecosistema original.



La idea central es sencilla:

`iasi.quarto` organiza y prepara. Quarto renderiza.

2.1 Qué hace Quarto

Quarto es el motor de publicación.

Entre otras tareas, se encarga de:

- interpretar los documentos `.qmd`;
- aplicar la configuración definida en `_quarto.yml`;
- ejecutar el código incluido en los documentos;
- resolver referencias, citas, figuras y tablas;
- aplicar temas, plantillas y estilos;
- generar formatos como HTML y PDF.

Un proyecto utilizado por `iasi.quarto` continúa siendo un proyecto Quarto y puede renderizarse directamente con sus herramientas habituales:

```
quarto render
```

`iasi.quarto` no altera ese principio.

2.2 Qué añade `iasi.quarto`

En un libro Quarto, la estructura de la publicación se declara normalmente en `_quarto.yml`.

Por ejemplo:

```
book:
  chapters:
    - index.qmd
    - chapters/01-intro.qmd
    - chapters/02-installation.qmd
    - chapters/03-usage.qmd
```

Esta configuración es clara y suficiente para publicaciones pequeñas. Sin embargo, a medida que un proyecto crece, mantener manualmente la relación de capítulos, partes y documentos puede convertirse en una tarea repetitiva y propensa a errores.

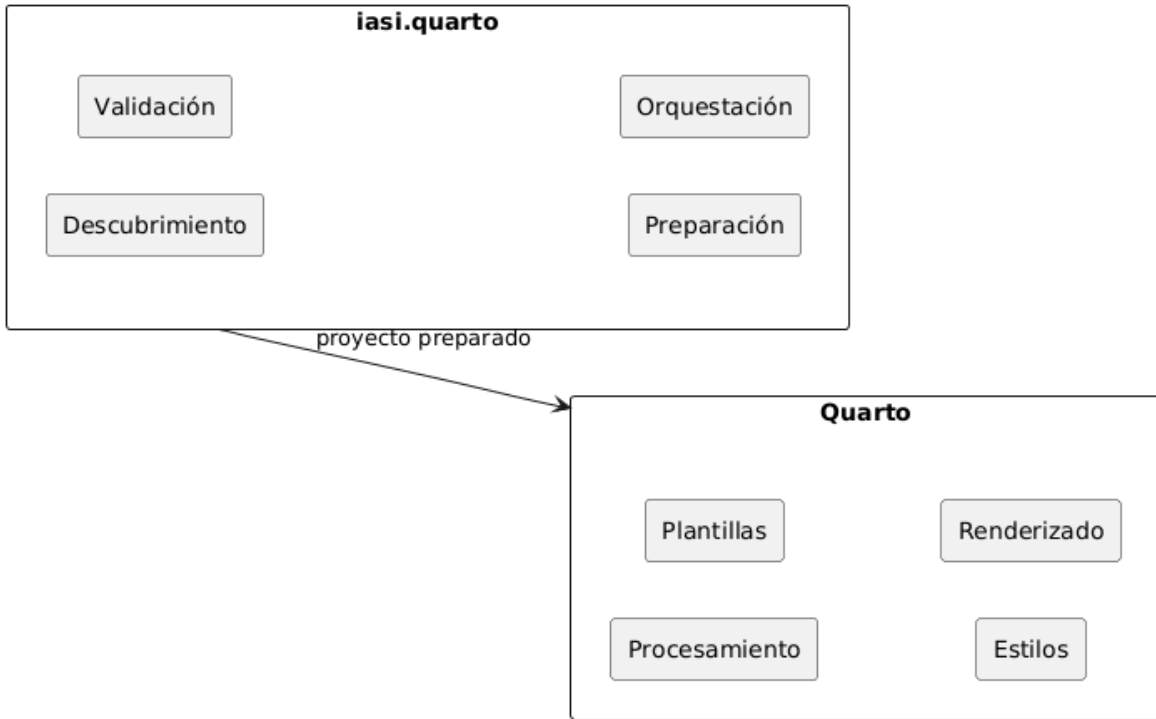
`iasi.quarto` añade una capa de convenciones y automatización que permite deducir parte de esa estructura a partir de la organización del proyecto.

Su trabajo consiste en:

- descubrir los documentos que forman la publicación;
- interpretar su organización;
- comprobar que la estructura sea coherente;
- preparar la configuración que necesita Quarto;
- coordinar la generación de uno o varios formatos.

2.3 Separación de responsabilidades

La separación entre ambas herramientas es deliberada.



Quarto controla:

- el modelo documental;
- los formatos de salida;
- los motores de ejecución;
- las plantillas;
- los estilos;
- el proceso de renderizado.

`iasi.quarto` controla:

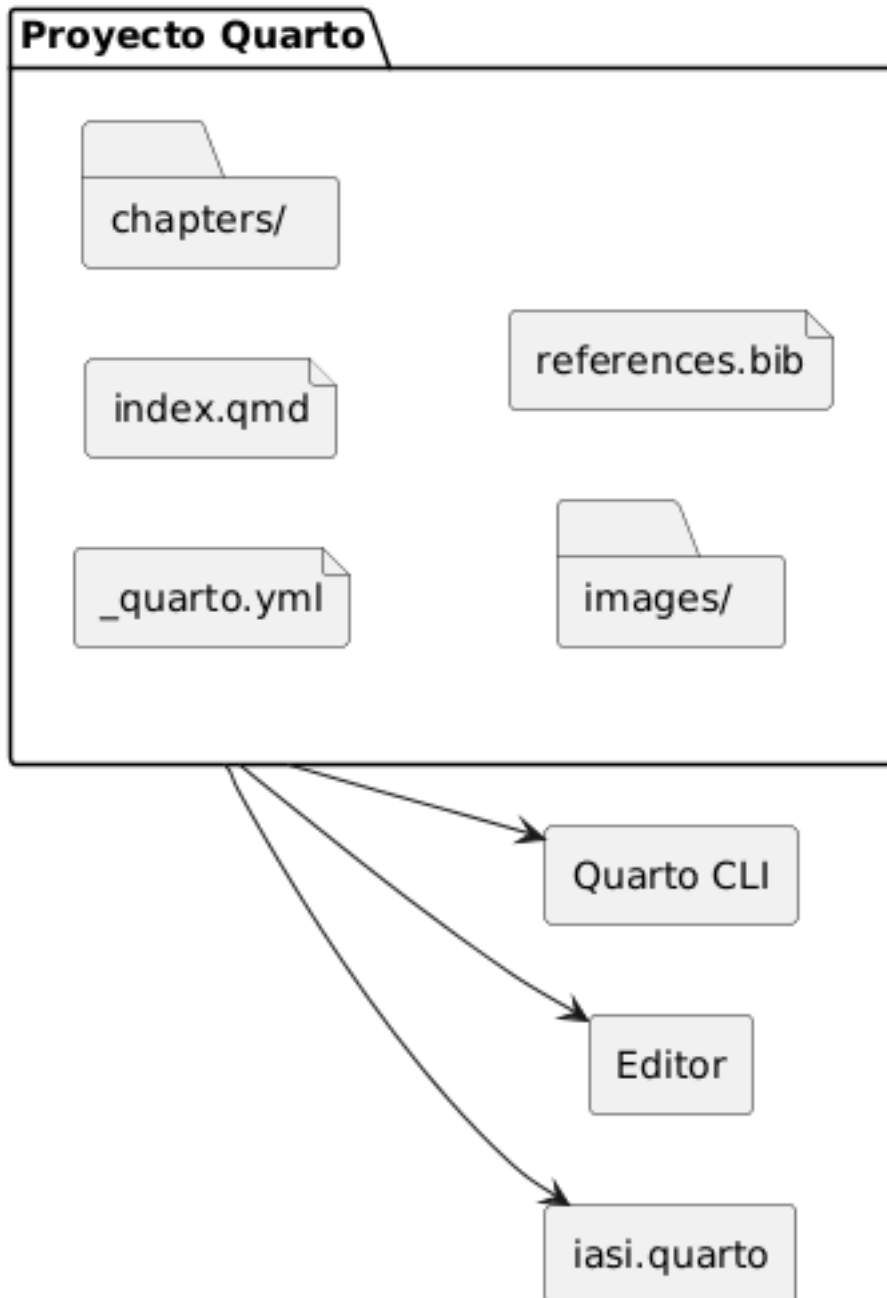
- las convenciones de organización;
- el descubrimiento de contenido;
- la validación de la estructura;
- la preparación de la publicación;
- la coordinación de libros y formatos.

Esta división evita duplicar capacidades que Quarto ya resuelve correctamente.

2.4 Un proyecto sigue siendo un proyecto Quarto

Adoptar `iasi.quarto` no encierra la publicación en un formato propietario.

El resultado preparado continúa utilizando archivos reconocibles por Quarto:



Esto permite:

- utilizar directamente la línea de comandos de Quarto;
- trabajar con las extensiones habituales del editor;
- conservar temas y configuraciones existentes;
- integrar otras extensiones de Quarto;
- dejar de utilizar `iasi.quarto` sin convertir los documentos.

`iasi.quarto` añade una capa sobre el proyecto, pero no se apropia de él.

2.5 Cuándo resulta útil

La automatización aporta más valor cuando:

- la publicación contiene muchos capítulos;
- los documentos cambian de posición con frecuencia;
- existen varios libros en un mismo repositorio;
- deben generarse varios formatos;
- se desea aplicar una organización uniforme;
- el proyecto debe poder crecer sin mantener manualmente listas extensas.

En publicaciones pequeñas, la ventaja principal es la sencillez.

En publicaciones grandes, la ventaja es mantener bajo control una estructura que, de otro modo, acabaría dispersa entre directorios, nombres de archivos y configuración YAML.

En el siguiente capítulo veremos cómo instalar `iasi.quarto` y comprobar que Quarto y R están correctamente disponibles.

3 Qué es `iasi.quarto`

`iasi.quarto` es una capa de organización y automatización construida sobre Quarto.

Su propósito no es sustituir el modelo de publicación de Quarto, sino facilitar el trabajo cuando un proyecto empieza a crecer: aparecen más capítulos, más formatos, distintas estructuras documentales y la necesidad de repetir tareas de preparación y renderizado de forma predecible.

3.1 Una convención sobre Quarto

Quarto ya sabe transformar documentos en libros, sitios web, artículos y otros formatos. `iasi.quarto` se ocupa de lo que ocurre alrededor de ese proceso:

- descubrir la estructura del proyecto;
- validar que esa estructura sea coherente;
- preparar los artefactos necesarios;
- renderizar uno o varios formatos;
- coordinar la construcción de uno o varios libros.

Quarto continúa siendo el motor de publicación. `iasi.quarto` actúa como capa de ingeniería.

3.2 El problema que resuelve

En un proyecto pequeño, mantener manualmente `_quarto.yml` puede ser suficiente. Cuando el número de documentos aumenta, esa configuración empieza a concentrar demasiado conocimiento:

- qué archivos forman parte del libro;
- en qué orden deben aparecer;
- cómo se organizan los capítulos;
- qué perfiles existen;
- qué formato debe generarse;
- dónde se escriben los resultados.

`iasi.quarto` convierte parte de ese conocimiento implícito en convenciones verificables.

La estructura del proyecto deja de ser una colección de decisiones manuales y pasa a ser información que el framework puede descubrir, validar y preparar.

3.3 Qué aporta

El flujo fundamental de `iasi.quarto` se divide en cuatro pasos:

```
project = discover()
validation = validate(project)
publication = prepare(validation)
render(publication)
```

En el uso habitual, `build()` coordina ese proceso completo.

Cada etapa tiene una responsabilidad concreta:

- `discover()` interpreta la estructura existente;
- `validate()` comprueba que el proyecto sea coherente;
- `prepare()` genera la configuración derivada necesaria;
- `render()` delega en Quarto la producción de los formatos finales.

Esta separación permite detectar los problemas antes de iniciar un renderizado y hace que cada transformación sea explícita.

3.4 Qué no pretende ser

`iasi.quarto` no es:

- un sustituto de Quarto;
- un nuevo lenguaje documental;
- un generador de contenido;
- una plantilla cerrada;
- una abstracción que oculte por completo la configuración de Quarto.

El proyecto sigue siendo un proyecto Quarto normal. Sus archivos pueden abrirse, editarse y renderizarse con las herramientas habituales.

La diferencia es que ciertas decisiones estructurales se expresan mediante convenciones que `iasi.quarto` puede comprender.

3.5 Una herramienta para proyectos que crecen

El valor de `iasi.quarto` aparece cuando la documentación deja de ser un conjunto de archivos aislados y empieza a comportarse como un sistema.

En ese momento importan el orden, la consistencia, la repetibilidad y la capacidad de modificar la estructura sin reconstruir manualmente toda la configuración.

`iasi.quarto` proporciona ese armazón sin quitarle a Quarto el papel central que le corresponde.

4 Fundamentos

4.1 La publicación IASI Quarto

Una **publicación IASI Quarto** es un proyecto Quarto cuya estructura, configuración y proceso de construcción siguen las convenciones definidas por `iasi.quarto`.

No es un tipo diferente de documento. Sigue siendo una publicación Quarto y puede utilizar sus formatos, extensiones y mecanismos habituales. `iasi.quarto` añade una capa de organización y automatización sobre ese proyecto:

- reconoce publicaciones;
- interpreta su estructura;
- comprueba su coherencia;
- genera los artefactos derivados;
- coordina su construcción.

La publicación continúa perteneciendo a Quarto. `iasi.quarto` no sustituye su motor, sino que organiza todo lo que ocurre antes de invocarlo.

4.1.1 La firma de una publicación

Una carpeta se reconoce como publicación IASI Quarto cuando contiene estos cuatro archivos:

```
_quarto.yml
_quarto-html.yml
_quarto-pdf.yml
iasi.yml
```

Los cuatro forman la **firma de la publicación**.

`_quarto.yml` contiene la configuración común del proyecto Quarto.

`_quarto-html.yml` contiene la configuración específica del perfil HTML.

`_quarto-pdf.yml` contiene la configuración específica del perfil PDF.

`iasi.yml` declara cómo debe interpretar y preparar la publicación `iasi.quarto`.

La coexistencia de los cuatro archivos es intencionada. La presencia aislada de `_quarto.yml` únicamente identifica un proyecto Quarto. La presencia aislada de `iasi.yml` tampoco es suficiente. Solo la firma completa identifica una publicación IASI Quarto.

Una estructura mínima puede ser:

```
my-publication/  
  _quarto.yml  
  _quarto-html.yml  
  _quarto-pdf.yml  
  iasi.yml  
  index.qmd  
  chapters/
```

La firma permite reconocer el proyecto sin analizar todavía toda su configuración ni recorrer su contenido. El reconocimiento es deliberadamente superficial: primero se determina qué existe; después se estudia qué contiene.

4.1.2 Una publicación no es necesariamente un libro

`iasi.quarto` utiliza el término **publicación** para referirse a la unidad que puede comprobarse, prepararse y construirse.

Actualmente, el caso principal es un libro Quarto:

```
project:  
  type: book
```

Sin embargo, la publicación y el tipo de proyecto son conceptos diferentes.

La publicación es la unidad gestionada por `iasi.quarto`.

El tipo indica cómo entiende Quarto ese proyecto:

```
book  
website  
manuscript  
article
```

Esta separación evita incorporar en la arquitectura una equivalencia innecesaria:

```
publicación libro
```

Un libro es un tipo posible de publicación. La publicación es el concepto más general.

4.1.3 Publicación individual

Cuando la carpeta indicada contiene directamente la firma completa, se considera una **publicación individual**.

Por ejemplo:

```
user-guide/  
  _quarto.yml  
  _quarto-html.yml  
  _quarto-pdf.yml  
  iasi.yml  
  index.qmd  
  chapters/
```

En este caso, la carpeta `user-guide/` es la raíz de la publicación.

La construcción se realiza desde esa raíz y todos los archivos de configuración, contenido y salida se interpretan en relación con ella.

```
validate("user-guide")
```

También puede validarse desde el propio directorio:

```
validate()
```

4.1.4 Multiproyecto

Una carpeta puede actuar como contenedor de varias publicaciones independientes.

```
docs/  
  user-guide/  
    _quarto.yml  
    _quarto-html.yml  
    _quarto-pdf.yml  
    iasi.yml
```



```
index.qmd
  chapters/

developer-guide/
  _quarto.yml
  _quarto-html.yml
  _quarto-pdf.yml
  iasi.yml
  index.qmd
  chapters/
```

La carpeta `docs/` no es una publicación. Es un **multiproyecto** que contiene dos publicaciones:

```
user-guide
developer-guide
```

Cada una conserva su propia configuración, estrategia, contenido y perfiles de salida.

El multiproyecto permite construirlas conjuntamente sin convertirlas en partes de un único libro.

```
validate("docs")
```

La validación reconoce el contenedor y localiza las publicaciones que contiene.

La raíz del multiproyecto no necesita disponer de su propio `iasi.yml`. Su función es agrupar publicaciones completas, no comportarse como una publicación adicional.

4.1.5 La publicación actual tiene prioridad

Cuando la carpeta indicada contiene la firma completa, se interpreta como una publicación individual aunque existan otras firmas en directorios descendientes.

```
publication/
  _quarto.yml
  _quarto-html.yml
  _quarto-pdf.yml
  iasi.yml
  index.qmd
  chapters/
```

```
examples/  
  sample-book/  
    _quarto.yml  
    _quarto-html.yml  
    _quarto-pdf.yml  
    iasi.yml
```

En este caso, `publication/` es la publicación actual. La publicación situada bajo `examples/` no se incorpora automáticamente al mismo plan de construcción.

Esta regla evita que ejemplos, plantillas, pruebas o proyectos auxiliares se confundan con publicaciones hermanas.

El reconocimiento sigue este orden:

1. comprobar si la carpeta actual contiene la firma completa;
2. si la contiene, tratarla como una publicación individual;
3. si no la contiene, buscar publicaciones completas en sus descendientes;
4. si se encuentran, tratar la carpeta como multiproyecto;
5. si no se encuentra ninguna, concluir que no es un proyecto IASI Quarto.

4.1.6 Publicación y contenido

La raíz de la publicación no obliga a utilizar un directorio llamado `chapters`.

El directorio de contenido se declara en `iasi.yml`:

```
publication:  
  content-dir: chapters
```

Podría utilizarse otro nombre:

```
publication:  
  content-dir: content
```

Y la estructura correspondiente sería:

```
my-publication/  
  _quarto.yml  
  _quarto-html.yml  
  _quarto-pdf.yml  
  iasi.yml
```

```
index.qmd
content/
```

La publicación es la raíz completa del proyecto. El directorio de contenido es solo una de sus partes.

Esta distinción permite que una publicación contenga, además del texto principal:

```
images/
resources/
bibliography/
extensions/
scripts/
outputs/
```

`iasi.quarto` no redefine toda la estructura de Quarto. Solo necesita conocer dónde se encuentra el contenido que debe organizar.

4.1.7 Archivos fuente y archivos derivados

Dentro de una publicación existen dos clases de archivos.

Los **archivos fuente** pertenecen al autor:

```
index.qmd
iasi.yml
_quarto.yml
_quarto-html.yml
_quarto-pdf.yml
chapters/01-intro/01-introduction.qmd
```

Los **archivos derivados** son generados por `iasi.quarto` durante la preparación:

```
_book-structure.yml
chapters/01-intro/index.qmd
```

La lista exacta depende de la estrategia de publicación.

La diferencia es importante:

- los archivos fuente expresan decisiones editoriales;
- los archivos derivados materializan esas decisiones para Quarto.

Un archivo derivado puede regenerarse. No debe convertirse accidentalmente en la única copia de contenido escrito por el autor.

4.1.8 Reconocer no significa construir

El reconocimiento de una publicación no implica que sea válida ni que pueda construirse.

Una carpeta puede contener la firma completa y, aun así, presentar problemas:

```
iasi.yml mal formado
estrategia desconocida
directorio de contenido inexistente
index.qmd raíz ausente
estructura incompatible con la estrategia
```

Por eso `validate()` responde a una pregunta limitada:

¿Existe aquí una publicación IASI Quarto o un multiproyecto reconocible?

No genera archivos, no modifica el proyecto y no ejecuta Quarto.

```
validate()
```

La construcción completa corresponde a:

```
build()
```

Esta separación permite inspeccionar un proyecto antes de intervenir sobre él.

4.1.9 La publicación como unidad de construcción

Una publicación reúne todo lo necesario para ejecutar una construcción independiente:

```
configuración común
configuración por formato
declaración IASI
contenido
artefactos derivados
salidas
```

En un multiproyecto, cada publicación mantiene esa independencia.

Esto permite construir:

- todas las publicaciones;
- una publicación concreta;
- todos los formatos;
- un formato determinado.

Por ejemplo:

```
build(  
  book = "user-guide",  
  format = "html"  
)
```

La selección pertenece al proceso de construcción. La publicación, por su parte, continúa siendo una unidad completa y autosuficiente.

Esa es la frontera fundamental del modelo:

Una publicación IASI Quarto es una unidad Quarto reconocible, configurable y construible de forma independiente.